

## المحاضرة الثالثة والرابعة: السلاسل الحرفية (المحرفية) والملفات النصية

### ملخص

تحتاج العديد من التطبيقات إلى التعامل مع السلاسل الحرفية: صياغة سلسلة، البحث ضمنها، تقطيعها.

تتضمن بايثون على كل ما يلزم من العمليات على السلاسل الحرفية. الملفات النصية هي وسيلة الخزن الدائم المعيارية/ القياسية لتبادل المعلومات بين مختلف البرامج، تتعامل بايثون مع الملفات التسلسلية ببساطة وسلاسة.

### أهداف تعليمية

يتعرف الطالب في هذا الفصل على:

- السلاسل الحرفية وأهم العمليات والطرائق عليها.
- الملفات النصية: كتابتها وقراءتها.

يمكن الطالب من:

من كتابة برامج بلغة بايثون تستخدم السلاسل الحرفية والملفات النصية.

## Variables

- Variable names تسمية المتغيرات
  - Valid identifier
    - Series of characters (letters, digits, underscores)
    - سلسلة من المحارف (احرف- ارقام- وصلة سفلية)
    - Cannot begin with digit
    - اسم المتغير لا يبدأ برقم ولا يحوي فراغات ولا أحرف خاصة ولا عمليات حسابية
    - Case sensitive
    - اللغة حساسة لحالة الاحرف (صغيرة – كبيرة)
    - يفضل أن يكون الاسم متناسبا مع طبيعة الاستخدام

## Decision Making: Equality and Relational Operators

| Standard algebraic equality operator or relational operator | Python equality or relational operator | Example of python condition | Meaning of Python condition                   |
|-------------------------------------------------------------|----------------------------------------|-----------------------------|-----------------------------------------------|
| <i>Relational operators</i>                                 |                                        |                             |                                               |
| >                                                           | >                                      | <b>x &gt; y</b>             | <b>x</b> is greater than <b>y</b>             |
| <                                                           | <                                      | <b>x &lt; y</b>             | <b>x</b> is less than <b>y</b>                |
| ≥                                                           | >=                                     | <b>x &gt;= y</b>            | <b>x</b> is greater than or equal to <b>y</b> |
| ≤                                                           | <=                                     | <b>x &lt;= y</b>            | <b>x</b> is less than or equal to <b>y</b>    |
| <i>Equality operators</i>                                   |                                        |                             |                                               |
| =                                                           | ==                                     | <b>x == y</b>               | <b>x</b> is equal to <b>y</b>                 |
| ≠                                                           | !=                                     | <b>x != y</b>               | <b>x</b> is not equal to <b>y</b>             |

السلاسل المحرفية: هي مجموعة من الاحرف والارقام والرموز ... محصورة بين علامتي تنصيص مفردة او مزدوجة " " أو ' ' تأخذ دليل تواجد كل حرف اتوماتيكياً عند التصريح عنها كمتحول محرفي (اما من البداية 0,1,2,... أو من النهاية -1,-2,-3,...

## STRINGS

|     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|----|----|----|----|----|----|----|----|----|
| -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
| H   | e   | l  | l  | o  |    | W  | o  | r  | l  | d  |
| 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |

- square brackets used to perform **indexing** into a string to get the value at a certain index/position

```
s = "abc"
```

index: 0 1 2 ← indexing always starts at 0

index: -3 -2 -1 ← last element always at index -1

s[0] → evaluates to "a"

s[1] → evaluates to "b"

s[2] → evaluates to "c"

s[3] → trying to index out of bounds, error

s[-1] → evaluates to "c"

s[-2] → evaluates to "b"

s[-3] → evaluates to "a"

يمكنك تقسيم السلسلة والتعامل مع جزء منها حسب ما تريد باستخدام الدليل Index و النقطتين العلويتين :

## STRINGS

البداية

النهاية

الخطوة

- can **slice** strings using [start:stop:step]
- if give two numbers, [start:stop], step=1 by default
- you can also omit numbers and leave just colons

```
s = "abcdefgh"
```

s[3:6] → evaluates to "def", same as s[3:6:1]

s[3:6:2] → evaluates to "df"

s[::] → evaluates to "abcdefgh", same as s[0:len(s):1]

s[::-1] → evaluates to "hgfedcba", same as s[-1:- (len(s)+1) :-1]

s[4:1:-2] → evaluates to "ec"

If unsure what some command does, try it out in your console!

## 1. السلاسل الحرفية Strings

يجري التصريح عن السلاسل الحرفية في بايثون عن طريق عملية الإسناد بإحاطة السلسلة النصية بالمحرفين ( ' الفاصلة العلوية، " الفاصلة العلوية المزدوجة - علامة التنصيص ) 'string' أو "string".

```
newString = "This is a string literal"
literalOne = "\\\MySystem\\MyDirectory\\ProgrammingC#.cs"
verbatimLiteralOne = r"\\MySystem\MyDirectory\ProgrammingC#.cs"
literalTwo = " Line One \n Line Two"
```

- نستخدم المحرف الخاص (\) للدلالة على أن المحرف التالي يجب أن يُعتبر كما هو وليس محرف خاص.
- نستخدم المحرف (r) لإلغاء جميع المحارف الخاصة ضمن السلسلة.
- إن (\n) تعني سطر جديد.

### ملاحظات:

3. للوصول إلى محرف في السلسلة النصية نستخدم الدليل str[i] (كما في القوائم list).
4. السلسلة المحرفية في بايثون غير قابلة للتعديل immutable أي أن العبارة str[i] = x تولّد خطأ.
5. نستخدم الأدلة في السلاسل الحرفية كما نستخدم في القوائم list.

### 1.1 صياغة السلسلة المحرفية

لدينا عدّة طرق لصياغة السلاسل المحرفية:

1. طريقة الجمع (ضمّ السلاسل): باستخدام العملية "+" بين السلاسل المحرفية لتشكيل سلسلة واحدة مثال:

```
newString = "Jon has " + str (34) + " Books "
```

2. الطريقة format المعرفة على النمط string: وهي تشبه طريقة الكتابة باستخدام قالب في C#، حيث كنا نستخدم "{0} {1}" .

مثال:

```
newString1 = "Jon has {} books".format(34)
newString2 = "Jon has {x} books".format(x=34)
newString3 = "Jon has {0} books and {1} pen".format(34, 45)
```

```
x = 123
print("The value is {:6d}".format(x))
print("The value is {:06d}".format(x))

x = 123.4567
print("The value is {:12.6f}".format(x))
print("The value is {:12,.3f}".format(x))
```

```
The value is      123
The value is 000123
The value is      123.456700
The value is      123.457
```

3. الطريقة المشابهة لطريقة printf في بعض لغات C-like (استخدام %):

مثال:

```
newString1 = "Jon has %d books"%(34)
newString3 = "Jon has %d books and %d pen"%(34, 45)
```

مثال آخر:

```
x = 123
print("The value is %6d"%(x))
print("The value is %06d"%(x))

x = 123.4567
print("The value is %12.6f"%(x))
print("The value is %12.3f"%(x))
```

```
The value is      123
The value is 000123
The value is      123.456700
The value is      123.457
```

## 2.1. أهم الطرائق للتعامل مع السلاسل الحرفية

يُبين الجدول التالي أهم الطرائق والعمليات المتوفرة للتعامل مع السلاسل النصية في بايثون:

| الطريقة أو العملية                        | الشرح                                                                                                                    |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>==, &lt;, &lt;=, &gt;, &gt;=</code> | تستخدم للمقارنة بين السلاسل (ترتيب كما في القواميس)، كما تأخذ بعين الاعتبار حالة الحرف (كبير مثل "A" صغير مثل "a")       |
| <code>+</code>                            | جمع/ ضمّ سلسلتين نصيتين                                                                                                  |
| <code>str1.endswith(str2)</code>          | اختبار انتهاء السلسلة بسلسلة أخرى                                                                                        |
| <code>is</code>                           | تستخدم للمقارنة بين السلاسل: تعيد True إذا تساوت السلسلتين                                                               |
| <code>str1.index("1")</code>              | فهرس أول ظهور لسلسلة ضمن سلسلة ابتداءً من فهرس معين أو <a href="#">ValueError: substring not found</a> في حال عدم الظهور |

|                                                                                                                   |                                                                         |
|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>str1[:3] + " in " + str1[3:]</code>                                                                         | إدراج سلسلة ضمن السلسلة اعتباراً من فهرس معين                           |
| <code>str1.rindex("1")</code>                                                                                     | فهرس آخر ظهور لسلسلة                                                    |
| <code>len(str1)</code>                                                                                            | عدد محارف السلسلة النصية                                                |
| <code>str1.split(sep=" ")</code><br><code>str1.split(sep=",")</code>                                              | تقوم بتقطيع سلسلة إلى مجموع سلاسل وفقاً لحرف فصل: مثل الفراغ أو الفاصلة |
| <code>str1.startswith("1")</code>                                                                                 | اختبار ابتداء السلسلة بسلسلة أخرى                                       |
| <code>str1[3:]</code><br><code>str1[:7]</code><br><code>str1[3:7]</code>                                          | الحصول على سلسلة جزئية من السلسلة                                       |
| <code>str1.lower()</code>                                                                                         | التحويل لأحرف صغيرة                                                     |
| <code>str1.upper()</code>                                                                                         | التحويل لأحرف كبيرة                                                     |
| <code>str1.strip()</code>                                                                                         | حذف الفراغات الزائدة من بداية ونهاية السلسلة                            |
| <code>str1.rstrip()</code>                                                                                        | حذف الفراغات الزائدة من نهاية السلسلة                                   |
| <code>str1.lstrip()</code>                                                                                        | حذف الفراغات الزائدة من بداية السلسلة                                   |
| <code>str4 =</code><br><code>"",".join(["a","b","c"])</code><br><code>#Print Output:</code><br><code>b,a,c</code> | دمج عناصر متحوّل من نمط تجميع عناصر (tuple, list)                       |

- يُستخدَم التابع (`str`) مع متحوّلات من أي نمط ويقوم بتحويلها إلى سلسلة حرفية. مثلاً:

```
m = 3.1416 ;
mString = str(m)
```

### 3.1. مثال تعليمي 1

يُبين المثال التالي استخدام بعض طرائق والعمليات على السلاسل النصية:

```
#create some strings to work with
s1 = "abcd"; s2 = "ABCD"
s3 = r"Anaconda Platform, provides custom PYTHON development,
on-site Training and Consulting;"
#result hold the results of comparisons
#compare two strings, case sensitive
result = (s1 == s2)
print("s1: %s == s2: %s, result: %s\n"%(s1, s2, result))
result = s1 > s2
print("s1: %s > s2: %s, result: %s\n"%(s1, s2, result))
result = s1 < s2
print("s1: %s < s2: %s, result: %s\n"%(s1, s2, result))
```

```
result = (s1.lower() == s2.lower())
print("compare insensitive\n")
print("s1: %s == s2: %s, result: %s\n"%(s1, s2, result))
result = s1.lower() > s2.lower()
print("s1: %s > s2: %s, result: %s\n"%(s1, s2, result))
```

يكون ناتج التنفيذ:

```
s1: abcd == s2: ABCD, result: False
s1: abcd > s2: ABCD, result: True
s1: abcd < s2: ABCD, result: False
compare insensitive
s1: abcd == s2: ABCD, result: True
s1: abcd > s2: ABCD, result: False
s1: abcd < s2: ABCD, result: False
```

```
#concatenation
s4 = s1 + s2
print("s6 concatenated from s1 + s2: ", s4)
#copy string
s5 = s4
print("s5 = s4: ", s5)
#Two ways to compare .
print("\nDoes s5 == s4?: ", s5 == s4)
print("\nDoes s5 is s4?: ", s5 is s4)
#Two useful methods: the index and the len
print("\nString s5 is %d characters long."%(len(s5)))
print("The 5th character is %s\n"%(s5[4]))
#test whether a string ends with a set of characters
print("s3:%s\nEnds with Training?:
%d\n"%(s3,s3.endswith("Training")))
print("Ends with Consulting?: %d"%( s3.endswith("Consulting"))))
#return the index of the substring
print("\nThe first occurrence of Training ")
print("in s3 is %s\n"%(s3.index("Training")))
#insert the word excellent before "training"
i = s3.index("Training")
s6 = s3[:i] + "excellent " + s3[i:]
print("s6: %s\n "%(s6))
```

يكون ناتج التنفيذ:

```
s6 concatenated from s1 + s2:  abcdABCD
s5 = s4:  abcdABCD

Does s5 == s4?:  True

Does s5 is s4?:  True

String s5 is 8 characters long .
The 5th character is A

s3:Anaconda Platform, provides custom PYTHON development, on-
site Training and Consulting
Ends with Training?: 0

Ends with Consulting?: 1

The first occurrence of Training
in s3 is 63

s6:  Anaconda Platform, provides custom PYTHON development, on-
site excellent Training and Consulting
```



## 4.1. مثال تعليمي 2

يُبين المثال التالي طرائق البحث ضمن السلاسل النصية:

```
s1 = "One Two Three Four"
#get the index of the last space
ix = s1.rindex(" ")
#get the last word.
s2 = s1[ix + 1:]
#set s1 to the substring starting at 0
#and ending at ix (the start of the last word)
#thus s1 has one two three
s1 = s1[0:ix]
#find the last space in s1 (after two)
ix = s1.rindex(" ")
#set s3 to the substring starting at
#ix, the space after "two" plus one more
#thus s3 = "three"
s3 = s1[ix + 1:]
print("s2: {0}\ns3: {1}".format(s2, s3))
#reset s1 to the substring starting at 0
#and ending at ix, thus the string "one two"
s1 = s1[0:ix]
#reset ix to the space between "one" and "two"
ix = s1.rindex(" ")
#set s4 to the substring starting one space after ix ,
#thus the substring "two"
s4 = s1[ix + 1:]
#reset s1 to the substring starting at 0 and ending at ix ,
#thus "one"
s1 = s1[0:ix]
#set s5 to the remaining substring in s1.
s5 = s1
print("s4: {0}\ns5: {1}\n".format(s4, s5))
print("s1: {0}\n".format(s1))
```

يكون ناتج التنفيذ:

```
s2: Four
s3: Three
s4: Two
s5: One

s1: One
```

### 5.1. مثال تعليمي3: مثال ذو فائدة عملية مباشرة

تُمكننا تعليمة القراءة `input()` فقط من إدخال سطر كسلسلة حرفية (يجب أن نضغط زر الإدخال Enter كي تتم القراءة)، ولنتذكر معاً أننا نقرأ الدخول كسطر كامل كسلسلة حرفية.

مثال: سطر الإدخال فيه القيمة 12، تُفهم كسلسلة حرفية من الحرفين "1" و"2"، ولتحويلها إلى قيمة عددية نستخدم `int()` أو `float()`، كنا مضطرين أن نعطي كل قيمة دخل على سطر منفصل. ومنه عدم المرونة في إدخال المعطيات عندما تكون المعطيات متعددة ومتراصة. مثال ذلك إدخال رقم الطالب وعلاماته واسمه، فمن المنطقي أن ندخلها على نفس السطر. مثل:

20510 90 85 60 Sami

يقوم البرنامج التالي باستخدام الطريقة `split` لتقطيع كل سطر دخل إلى سلاسل نصية يفصلها الفراغ، ويحوّل كل سلسلة إلى نمط القيمة الموافقة، ثم يعطينا أكبر علامة للطالب. (في المثال ننهي الدخول بالسطر الحاوي للقيمة صفر).

```
#input line data example : 20510 85 60 80 Sami
id = -1
while id!=0:      #end input with zeros
    print(" Data: id, eng, math, prog, name :")
    line = input()
    items = line.split()
    id = int(items[0])
    if(id == 0):
        break
    else:
        eng = float(items[1])
        math = float( items[2])
        prog = float(items[3])
        name = items[4]
        print("{0}      {1}      Max=      {2}      ".format(id, name,
max(eng,math,prog)))
```

يكون ناتج التنفيذ:

```
Data: id, eng, math, prog, name:
20510  80 60 85 Sami
20510  Sami Max= 85.0
Data: id, eng, math, prog, name:
20511  70 23 34 Rama
20511  Rama Max= 70
Data: id, eng, math, prog, name:
0
```

## 6.1. فقط في بايثون: ضرب السلاسل الحرفية!

تؤدي عملية الضرب \* بين سلسلة string وعدد صحيح n إلى تكرار السلسلة n مرة. (جميل أن تأخذ عملية الضرب معناها الفعلي حتى مع السلاسل الحرفية: أليس n\*5 هو جمع العدد 5 n مرة. وكذلك سيكون '5'\*n هو جمع السلسلة '5' n مرة. مثال:

```
print("input k in [2..10] = ")
k=input()
# Multiplication Table of number k
i = 1
while (i <= 10):
    print(k , " x " , i , " = " , k * i)
    i = i+1
```

```
5 x 1 = 5
5 x 2 = 55
5 x 3 = 555
5 x 4 = 5555
5 x 5 = 55555
5 x 6 = 555555
5 x 7 = 5555555
5 x 8 = 55555555
5 x 9 = 555555555
5 x 10 = 5555555555
```

## 7.1. تمارين على السلاسل الحرفية

1. المطلوب كتابة برنامج يُمرر له سلسلة نصية فتقوم بتحويل الحرف الأول من كل كلمة إلى حرف

كبير. مثلاً من أجل الدخل: I go to school every day

يكون الخرج: I Go To School Every Day

2. المطلوب كتابة برنامج يستخدم الطريقة `index()` لحساب مواقع كلمة معينة ضمن سلسلة نصية وعدد

مرات ورود هذه الكلمة في هذه السلسلة.

تذكّر: `index` تعطي فهرس أول ظهور لسلسلة ضمن سلسلة إبتداءً من فهرس معين.

`index("am",0)` "I am..I am Happy". يعطي 2

`index("Am",0)` "I am..I am Happy". يعطي خطأ

[ValueError: substring not found](#)

`index("am",3)` "I am..I am Happy". يعطي 8

```

S="what is world wide web : www"
sf="w"
print(" String : " + S)
print(" Looking for  :" + sf)
print()
ib= iw =nw=0
while ib < len(S):
    iw = S.index(sf,ib)
    if (iw != -1 ):
        print( " {0} found at index {1}".format(sf, iw))
        nw += 1

    ib=iw+1
print("word ({0}) found {1} times ".format(sf, nw))

```

3. نريد إظهار الترميز Unicode للأحرف العربية (يبدأ ترميز العربية من العدد 1569).

```

s=""
for i in range(1569,1590):
    s += str(i) + " = " + chr(i) + "\n"
s = "Arabic Unicode\n" + s
print(s)

```

ويكون الخرج:

```

Arabic Unicode
ا = 1569
آ = 1570
أ = 1571
ؤ = 1572
إ = 1573
ئ = 1574
ا = 1575
ب = 1576
ة = 1577
ت = 1578
ث = 1579
ج = 1580
ح = 1581
خ = 1582
د = 1583
ذ = 1584
ر = 1585
ز = 1586
س = 1587
ش = 1588
ص = 1589

```

تمارين إضافية:

Out

```
print("بعض العمليات على السلاسل المحرفية")
name1="Hussain"; name2="ahmad"# كتابة اكثر من سطر معا بينهم

print("1- ",name1)
print("2- ",len(name1),len(name2))# طباعة عدد احرف سلسلة
print("3- ",name1[0],name2[2])# طباعة عنصر من السلسلة ذو دليل
print("4- ",name2.count('s'))# طباعة عدد مرات تكرار حرف ما
print("5- ",name1.find('i'))# بحث عن محرف في السلسلة
print("6- ",name1.count('s'))# طباعة عدد مرات تكرار محرف في سلسلة
print("7- ",name1.count('s'))# طباعة عدد مرات تكرار محرف في سلسلة
print("8- ",name2.capitalize())# تجميع الحرف الاول من السلسلة
print("9- ",name2.isdigit())# اختبار الاحرف اذا كانت ارقاما
print("10- ",name1.lower())# تحويل جميع احرف السلسلة الى صغيرة
print("11- ",name2.upper())# تحويل جميع احرف السلسلة الى كبيرة
rep=name1.replace("ss","fff")# استبدال سلسلة جزئية بأخرى
print("12- ",rep)# طباعة السلسلة الجديدة بعد الاستبدال
```

بعض العمليات  
على السلاسل  
المحرفية

1- Hussain  
2- 7 5  
3- H m  
4- 0  
5- 5  
6- 2  
7- 2  
8- Ahmad  
9- False  
10- hussain  
11- AHMAD  
12- Hufffain

افهم عمل كل سطر وحل النتيجة ثم قم بتغيير قيم المتغيرات المحرفية name1 و name2 ثم غير محارف البحث والاستبدال ... حتى تتمكن من التعامل القوي المرن مع السلاسل المحرفية والاستفادة من ذلك في عملك البرمجي

```
print("ادخال القيم من لوحة المفاتيح")
name = input("What is your name ? : ")# القيمة محرفية دوما باستخدام input
age = int(input(" How old are you ? : "))# قسر القيمة المدخلة الى صحيحة
height=float(input("How tall are you"))
age = age+1# integer يمكننا التعامل معه كمتغير صحيح
print("Hello "+name)# سلسلتين محرفيتين (جمع) يمكننا اضافة
print("You are ",age, "old")# لا نستطيع جمع سلسلة مع عدد صحيح
print("You are "+str(age), "old")# قسرنا العدد الصحيح الى سلسلة وجمعنا
print("You are "+str(height)+ " CM tall")# قسرنا العدد الحقيقي
```

ادخال القيم من لوحة المفاتيح  
What is your name ? : Nadia  
How old are you ? : 24  
How tall are you 168.5  
Hello Nadia  
You are 25 old  
You are 25 old  
You are 168.5 CM tall

## Working with strings

امثلة اضافية عن التعامل مع سلاسل الحروف

In: `test_string = "Dogs are better than cats"`  
`print('Last element:', test_string[4:])`  
`print('Second to last element:', test_string[:4])`  
 Out: Last element: are better than cats  
 Second to last element: Dogs

In: `test_string = "Dogs are better than cats"`  
`print(test_string[5:10])`  
 Out: are b

|     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|----|----|----|----|----|----|----|----|----|
| -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
| H   | e   | l  | l  | o  |    | W  | o  | r  | l  | d  |
| 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |

- Create a string that is 10 characters in length.
- Print the second character to the screen.
- Print the third to last character to the screen.
- Print all characters after the fourth character.
- Print characters 2-8.

## Working with strings

امثلة اضافية عن التعامل مع سلاسل الحروف

In: `greeting = 'Hello, Lew!'`  
`print('1:', greeting)`  
`print('2:', len(greeting))`  
`print('3:', greeting[0])`  
`print('4:', greeting[-1])`  
`greeting = greeting.replace("Lew", "class")`  
`print('5:', greeting)`  
`string1 = "Hello"`  
`string2 = "world"`  
`print("1:", string1, string2)`  
`cost = float(35.28)`  
`print("Bar tab = £%f" %cost)`  
 Out: 1: Hello, Lew!  
 2: 11  
 3: H  
 4: !  
 5: Hello, class!

1: Hello world  
 Bar tab = £35.280000

- Create a string variable.
- Work out the length of the string.

## التوابع الرياضية Mathematical Function

### استيراد مكتبة التوابع الرياضية import math

```
import math
x = min(5, 10, 25); print("1: ",x)
y = max(5, 10, 25) ; print("2: ",y)
a = abs(-7.25); print("3: ",a)
x = pow(4, 3); print("4: ",x)
x = math.sqrt(64); print("5: ",x)
x = math.ceil(1.7); print("6: ",x)
y = math.floor(1.4); print("7: ",y)
x = math.pi; print("8: ",x)
a = math.cos(0); print("9: ",a)
a = math.sin(math.pi/6); print("10: ",a)
a = math.e; print("11: ",a)
a = math.degrees(math.pi/2); print("12: ",a)
```

Out:

```
1: 5
2: 25
3: 7.25
4: 64
5: 8.0
6: 2
7: 1
8: 3.141592653589793
9: 1.0
10: 0.49999999999999994
11: 2.718281828459045
12: 90.0
```

حلل - استنتج - جرب - غير قيم - فكر متى نستخدم تابع ما وماذا يفيد -  
ابحث عن توابع رياضية اخرى

تمارين إضافية مع شروحات للمساعدة على التمرين:

str

Indexing | الفهرسة 1

```
course="Python programming"
```

Index

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 Positive indexing  
-18 -17 -16 -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1 Negative indexing

```
course[0] # Get "P"
course[4] # Get "o"
course[-1] # Get "g"
course[-6] # Get "a"
```

str

Slicing | التقطيع 2

```
course="Python programming"
```

Index

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 Positive indexing  
-18 -17 -16 -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1 Negative indexing

```
course[0:3] # Get "Pyt"
course[7:14] # Get "program"
course[:6] # Get "Python"
course[0:] # Get "Python programming"
```



## تمرين:

- (1) قم بتعريف متغير اسمه site و أسند له القيمة النصية "Wikipedia"
- (2) إستخرج الأحرف "k" و "d" ثم إطبعاها على الشاشة
- (3) إستخرج المقاطع "Wik" و "pedia" ثم إطبعاها على الشاشة
- (4) إستبدل الحرف "p" بالحرف "m" ليصبح النص "Wikimedia"

## مناقشة كود التمرين السابق:

```
python strings.py > ...
1 site = "Wikipedia"
2 print(site[2], site[6])
3 print(site[-7], site[-3])
4 print(site[0:3], site[:3], site[:-6])
5 print(site[4:], site[-5:])
6 # site[4] = "m"
```

ملاحظة: إن السلاسل النصية غير قابلة للتغيير (Immutable) لذلك تم حجب السطر السادس من الكود لأنه سيعطيك خطأ فما الحل برأيك؟ لاحظ التغيير في الكود التالي وناقش؟

```
python strings.py > ...
1 site = "Wikipedia"
2 print(site[2], site[6])
3 print(site[-7], site[-3])
4 print(site[0:3], site[:3], site[:-6])
5 print(site[4:], site[-5:])
6 site = site.replace("p", "m")
7 print(site)
```



العوامل المهمة في التعامل مع السلاسل الحرفية (المحرفية أو النصية):

## 1 | Concatenate operator

| Python Syntax | Example                                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>+</b>      | <pre>first_name = "Elon" last_name = "Musk" full_name = first_name + " " + last_name print(full_name) # Get "Elon Musk"</pre> |

## 2 | Repetition operator

| Python Syntax | Example                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------|
| <b>*</b>      | <pre>greeting = "Hello" long_greeting = greeting * 3 print(long_greeting) # Get "HelloHelloHello"</pre> |

## 3 | Membership operators

| Python Syntax | Example                                                                                                                                                                                                  |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>in</b>     | <pre>message = "Python is easy to learn" print("easy" in message) # Get True print("hard" in message) # Get False print("easy" not in message) # Get False print("hard" not in message) # Get True</pre> |
| <b>not in</b> |                                                                                                                                                                                                          |

بعض التوابع المستخدمة للتعامل مع السلاسل الحرفية (المحرفية أو النصية):

Fonctions

الدوال

Parameters  
الإعدادات

```
variable_name = function_name( parm1 , parm2 , ... )
```

e Example

```
site = "YouTube"  
print(type(site)) # Get str  
print(len(site)) # Get 7
```

قم بتجريب الكود التالي وناقش النتائج:

```
string functions.py > ...  
1 course = "python programming"  
2 print(len(course))  
3 print(type(course))  
4 print(course.upper())  
5 print(course.lower())  
6 print(course.title())  
7 print(course.strip())  
8 print(course.find("pro"))  
9 print(course.count("o"))  
10 print(course.replace("p", "j"))
```

## تمرين 1

- (1) قم بتعريف التغيرات التالية:
  - متغير اسمه s1 و أعطه القيمة النصية "Learn all about"
  - متغير اسمه s2 و أعطه القيمة النصية "python strings"
- (2) قم بتحويل السلسلة النصية s2 لتصبح "Python Strings"
- (3) قم بدمج القيم النصية للمتغيرين s1 و s2. خزن القيمة الجديدة في متغير s3
- (4) إطبّع عدد أحرف السلسلة النصية s3 على الشاشة
- (5) قم بتحديد مكان كلمة "Python" في السلسلة النصية s3
- (6) إحذف الكلمة "Strings" من النص s3

ناقش الكود الخاص بالتمرين السابق: (جرب تتعلم)

```
str exercise.py > ...  
1  s1 = "Learn all about"  
2  s2 = "python strings"  
3  s2 = s2.title()  
4  print(s2)  
5  s3 = s1 + " " + s2  
6  print(s3)  
7  print(len(s3))  
8  print(s3.find("Python"))  
9  print(s3.replace("Strings", ""))
```

## تنسيق السلاسل النصية String formatting

```
player_name = "Bilel"
score = 10
"player_name score"
```

 f-string

### Syntax

```
f" {variable} {expression} "
```

### Example

```
player_name = "Bilel"
score = 10
print(f"{player_name} has {score} points")
# Get Bilel has 10 points
```

ناقش الكود التالي وماذا سيكون خرجه؟

```
f-string.py > ...
1 name = "Ahmed"
2 age = 25
3 message = "Hello " + name + " you are " + str(age) + " years old"
4 print(message)
```

بعض أنواع أحرف الهروب الهامة:

## أحرف الهروب Escape characters

| Python Syntax | Description    |
|---------------|----------------|
| \'            | Single quote   |
| \"            | Double quotes  |
| \n            | New ligne      |
| \t            | Horizontal Tab |
| \\            | Back slash     |

ناقش الكود التالي وماذا تتوقع أن يكون الخرج على الشاشة؟ (جرب تتعلم)

```
escape_characters.py
1 print('It\'s the right time to learn Python')
2 print("It's the right time to learn \"Python\"")
3 print("What's your favorite code editor?\nVS Code")
4 print("\tPython course")
```



## التعامل مع الملفات File Handling



### 2. الملفات النصية التسلسلية

يُمكننا في هذا النوع من الملفات قراءة أو كتابة البيانات "نص" من بداية الملف إلى نهايته.

#### 1.2. الكتابة في ملف نصي تسلسلي

نحتاج للكتابة في ملف تسلسلي إلى تعريف متحول من نوع "ملف" مع خيار كتابة:

```
fw = open(path,'w')
```

- إن تعريف المتحول (fw) بهذا الشكل يؤدي إلى ربط (المتحول الملف) بالملف الحقيقي على القرص والمحدد مساره بالسلسلة الحرفية (path).
- كل كتابة على المتحول (fw) تجري فعلياً على الملف الحقيقي (path).
- إن كان الملف (path) موجود سابقاً يحذف محتواه.
- إن لم يكن الملف (path) موجود سابقاً يُخلَق ملف جديد بالمسار (path).
- للكتابة على الملف نستخدم:
  - الطريقة (fw.write("string")) التي تكتب السلسلة الحرفية إلى الملف.
  - الطريقة (fw.writelines(listOfLines)) التي تكتب قائمة من السلاسل الحرفية إلى الملف.

#### 2.2. القراءة من ملف نصي تسلسلي

نحتاج للقراءة من ملف تسلسلي إلى إنشاء متحول ملف مع خيار قراءة:

```
fr = open(path,'r')
```

- إن تعريف المتحول (fr) بهذا الشكل يؤدي إلى ربط (المتحول الملف) بالملف الحقيقي على القرص والمحدد مساره بالسلسلة الحرفية (path).
- كل قراءة من المتحول (fr) تجري فعلياً على الملف الحقيقي (path).
- للقراءة من الملف نستخدم:
  - الطريقة (fr.read()) والتي تقرأ كافة محتويات الملف إلى سلسلة حرفية.
  - الطريقة (fr.readline()) والتي تقرأ سطر واحد من الملف إلى سلسلة حرفية.
  - الطريقة (fr.readlines()) والتي تقرأ أسطر الملف وتخزنها إلى قائمة من السلاسل المحرّفة.

### 3.2. مثال

يُبين المثال التالي استخدام الملفات النصية للكتابة.

```
fw = open("filePath.txt", "w")
fw.writelines(["First line\n", "Second line\n", "Third line\n"])

fw.write("Hello World of text files!\n")
fw.write("This is a Python example of Text files\n")
fw.write("\n")
fw.close()
```

ويُبين المثال التالي استخدام الملفات النصية للقراءة.

```
fr = open(r"d:\filePath.txt", "r")

result = fr.readline()
print("1: ", result)
i=1
while result != "":
    i += 1
    result = fr.readline()
    print(str(i) + ": ", result)

fr.close()
```

ويكون الخرج كما يلي:

```
1: First line
2: Second line
3: Third line
4: Hello World of text files!
5: This is a Python example of Text files
6:
7:
```



## 4.2. قراءة الملف النصي دفعة واحدة

في معظم لغات البرمجة يكون التعامل مع الملفات النصية كما جرى شرحه عبر المثال.

- ربط المتحول/ الملف بالملف الحقيقي.
- قراءة كامل الملف بالشكل:

ما دام (لم نصل إلى نهاية الملف).  
اقرأ سطرًا من الملف.

مع التوسع الكبير في حجوز الذاكرة، صار بالإمكان وضع كل الملف النصي في الذاكرة المركزية. وهذا ما أتاحتها بايثون بسهولة بالغة. حيث يمكن باستدعاء واحد للطريقة `fr.read()` قراءة كامل الملف إلى سلسلة محرفية واحدة.

مثال: قراءة كامل الملف النصي التالي:

```
20510 85 60 80 Sami
20420 60 80 90 Rana
20600 95 70 50 Jamil
```

```
fr = open(r"d:\filePath.txt", "r")
result = fr.read()
print(result)
fr.close()
```

ويكون الخرج كما يلي:

```
20510 85 60 80 Sami
20420 60 80 90 Rana
20600 95 70 50 Jamil
```

قراءة كامل الملف كقائمة سلاسل حرفية:

بالطريقة `fr.readlines()` التي تقوم بقراءة كامل الملف إلى قائمة من السلاسل الحرفية دون قراءة الملف سطرًا سطرًا.

```
fr = open(r"d:\filePath.txt", "r")
result = fr.readlines()
for i in range(len(result)):
    print("%d: %s" % (i, result[i]))
fr.close()
```

ويكون الخرج كما يلي:

```
1: 20510 85 60 80 Sami
2: 20420 60 80 90 Rana
3: 20600 95 70 50 Jamil
```

**فقط في بايثون** يمكن الوصول إلى أسطر الملف `fr` مباشرة كما يبين المثال التالي الذي يعطي نفس الخرج السابق:

```
fr = open(r"d:\filePath.txt", "r")
i = 0
for line in fr:
    i = i + 1
    print("%d: %s" % (i, line))
fr.close()
```

### شروحات للمساعدة

| Text file modes |                                                          |
|-----------------|----------------------------------------------------------|
| Mode            | Description                                              |
| "r"             | Open for reading (default)                               |
| "w"             | Open for writing, truncating the file first              |
| "a"             | Open for writing, appending to the end of the file       |
| "r+"            | Open for reading and writing without truncating the file |
| "w+"            | Open for writing and reading, truncating the file first  |
| "a+"            | Open for appending and reading                           |

**انتهت المحاضرات الثالثة والرابعة**

## التوابع مسبقة التعريف المستعملة

*chr()*, *float()*, *int()*, *len()*, *max()*, *open()*,  
*ord()*, *str()*

## التوابع المسبقة التعريف في بايثون Built-in functions (عددتها نحو 70)

|                      |                    |                     |                   |                       |
|----------------------|--------------------|---------------------|-------------------|-----------------------|
| <i>abs()</i>         | <i>dict()</i>      | <i>help()</i>       | <i>min()</i>      | <i>setattr()</i>      |
| <i>all()</i>         | <i>dir()</i>       | <i>hex()</i>        | <i>next()</i>     | <i>slice()</i>        |
| <i>any()</i>         | <i>divmod()</i>    | <i>id()</i>         | <i>object()</i>   | <i>sorted()</i>       |
| <i>ascii()</i>       | <i>enumerate()</i> | <i>input()</i>      | <i>oct()</i>      | <i>staticmethod()</i> |
| <i>bin()</i>         | <i>eval()</i>      | <i>int()</i>        | <i>open()</i>     | <i>str()</i>          |
| <i>bool()</i>        | <i>exec()</i>      | <i>isinstance()</i> | <i>ord()</i>      | <i>sum()</i>          |
| <i>bytearray()</i>   | <i>filter()</i>    | <i>issubclass()</i> | <i>pow()</i>      | <i>super()</i>        |
| <i>bytes()</i>       | <i>float()</i>     | <i>iter()</i>       | <i>print()</i>    | <i>tuple()</i>        |
| <i>callable()</i>    | <i>format()</i>    | <i>len()</i>        | <i>property()</i> | <i>type()</i>         |
| <i>chr()</i>         | <i>frozenset()</i> | <i>list()</i>       | <i>range()</i>    | <i>vars()</i>         |
| <i>classmethod()</i> | <i>getattr()</i>   | <i>locals()</i>     | <i>repr()</i>     | <i>zip()</i>          |
| <i>compile()</i>     | <i>globals()</i>   | <i>map()</i>        | <i>reversed()</i> | <i>__import__()</i>   |
| <i>complex()</i>     | <i>hasattr()</i>   | <i>max()</i>        | <i>round()</i>    |                       |
| <i>delattr()</i>     | <i>hash()</i>      | <i>memoryview()</i> | <i>set()</i>      |                       |

| Function             | Description                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------|
| <b>abs()</b>         | Returns the absolute value of a number                                                        |
| <b>all()</b>         | Returns True if all items in an iterable object are true                                      |
| <b>any()</b>         | Returns True if any item in an iterable object is true                                        |
| <b>ascii()</b>       | Returns a readable version of an object. Replaces none-ascii characters with escape character |
| <b>bin()</b>         | Returns the binary version of a number                                                        |
| <b>bool()</b>        | Returns the boolean value of the specified object                                             |
| <b>bytearray()</b>   | Returns an array of bytes                                                                     |
| <b>bytes()</b>       | Returns a bytes object                                                                        |
| <b>callable()</b>    | Returns True if the specified object is callable, otherwise False                             |
| <b>chr()</b>         | Returns a character from the specified Unicode code.                                          |
| <b>classmethod()</b> | Converts a method into a class method                                                         |
| <b>compile()</b>     | Returns the specified source as an object, ready to be executed                               |
| <b>complex()</b>     | Returns a complex number                                                                      |
| <b>delattr()</b>     | Deletes the specified attribute (property or method) from the specified object                |
| <b>dict()</b>        | Returns a dictionary (Array)                                                                  |
| <b>dir()</b>         | Returns a list of the specified object's properties and methods                               |
| <b>divmod()</b>      | Returns the quotient and the remainder when argument1 is divided by argument2                 |
| <b>enumerate()</b>   | Takes a collection (e.g. a tuple) and returns it as an enumerate object                       |
| <b>eval()</b>        | Evaluates and executes an expression                                                          |
| <b>exec()</b>        | Executes the specified code (or object)                                                       |
| <b>filter()</b>      | Use a filter function to exclude items in an iterable object                                  |
| <b>float()</b>       | Returns a floating point number                                                               |
| <b>format()</b>      | Formats a specified value                                                                     |

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| <b>frozenset()</b>  | Returns a frozenset object                                                         |
| <b>getattr()</b>    | Returns the value of the specified attribute (property or method)                  |
| <b>globals()</b>    | Returns the current global symbol table as a dictionary                            |
| <b>hasattr()</b>    | Returns True if the specified object has the specified attribute (property/method) |
| <b>hash()</b>       | Returns the hash value of a specified object                                       |
| <b>help()</b>       | Executes the built-in help system                                                  |
| <b>hex()</b>        | Converts a number into a hexadecimal value                                         |
| <b>id()</b>         | Returns the id of an object                                                        |
| <b>input()</b>      | Allowing user input                                                                |
| <b>int()</b>        | Returns an integer number                                                          |
| <b>isinstance()</b> | Returns True if a specified object is an instance of a specified object            |
| <b>issubclass()</b> | Returns True if a specified class is a subclass of a specified object              |
| <b>iter()</b>       | Returns an iterator object                                                         |
| <b>len()</b>        | Returns the length of an object                                                    |
| <b>list()</b>       | Returns a list                                                                     |
| <b>locals()</b>     | Returns an updated dictionary of the current local symbol table                    |
| <b>map()</b>        | Returns the specified iterator with the specified function applied to each item    |
| <b>max()</b>        | Returns the largest item in an iterable                                            |
| <b>memoryview()</b> | Returns a memory view object                                                       |
| <b>min()</b>        | Returns the smallest item in an iterable                                           |
| <b>next()</b>       | Returns the next item in an iterable                                               |
| <b>object()</b>     | Returns a new object                                                               |
| <b>oct()</b>        | Converts a number into an octal                                                    |
| <b>open()</b>       | Opens a file and returns a file object                                             |

|                        |                                                                                 |
|------------------------|---------------------------------------------------------------------------------|
| <b>ord()</b>           | Convert an integer representing the Unicode of the specified character          |
| <b>pow()</b>           | Returns the value of x to the power of y                                        |
| <b>print()</b>         | Prints to the standard output device                                            |
| <b>property()</b>      | Gets, sets, deletes a property                                                  |
| <b>range()</b>         | Returns a sequence of numbers, starting from 0 and increments by 1 (by default) |
| <b>repr()</b>          | Returns a readable version of an object                                         |
| <b>reversed()</b>      | Returns a reversed iterator                                                     |
| <b>round()</b>         | Rounds a numbers                                                                |
| <b>set()</b>           | Returns a new set object                                                        |
| <b>setattr()</b>       | Sets an attribute (property/method) of an object                                |
| <b>slice()</b>         | Returns a slice object                                                          |
| <b>sorted()</b>        | Returns a sorted list                                                           |
| <b>@staticmethod()</b> | Converts a method into a static method                                          |
| <b>str()</b>           | Returns a string object                                                         |
| <b>sum()</b>           | Sums the items of an iterator                                                   |
| <b>super()</b>         | Returns an object that represents the parent class                              |
| <b>tuple()</b>         | Returns a tuple                                                                 |
| <b>type()</b>          | Returns the type of an object                                                   |
| <b>vars()</b>          | Returns the __dict__ property of an object                                      |
| <b>zip()</b>           | Returns an iterator, from two or more iterators                                 |